

APUNTES COMPLETOS DE PHP

Autor: Egoitz Guerras Rodríguez

Contenido

0. Definición, instalación, configuración y ejecución de PHP.....	5
0.1 Windows (XAMPP).....	5
0.2 Linux	5
0.3 Ejecución por terminal	6
1. Sintaxis PHP	6
Explicación línea por línea	6
1.1 Cierre de PHP	6
1.2 Mezclar PHP y HTML	6
2. Comentarios	7
3. Variables y tipos de datos.....	7
3.1 Variables simples.....	7
3.2 Tipos de Datos	7
3.2.1Ejemplo completo	7
3.3.2 Strings	8
3.4.2 Tipado Estricto.....	10
3.4.2.1 Ejemplo SIN strict_types.....	10
3.4.2.1 Ejemplo CON strict_types.....	10
5. Operadores.....	11
5.1 Operadores Aritméticos	11
5.2 Operadores de Comparación	11
5.3 Operadores lógicos.....	11
5.4 Operadores relacionales.....	11
5.5 Operador spaceship (PHP Moderno).....	11
5.6 Operadores de asignación combinada	12
5.7 Operador de concatenación	12
5.8 Operador null coalescing.....	12
5. Estructuras de control	13

APUNTES COMPLETOS DE PHP

5.1 if / else	13
5.2 switch.....	14
5.3 match (PHP 8+ -- alternativa a switch)	15
6. Bucles.....	16
6.1 for	16
6.2 While.....	17
6.3 foreach.....	18
7. Arrays.....	19
7.1 Sintaxis.....	19
7.2 Funciones del array	20
8. Funciones.....	22
8.1 Función basica	22
function saludar() {	22
8.2 Parámetros y retorno sin tipado	22
8.3 Parámetros y retorno con tipado	22
8.4 Funciones anónimas(closures)	23
9. Superglobales	24
9.1 \$_GET.....	24
9.2 \$_POST	24
9.3 \$_SESSION	25
9.3.1 Posicionar sesion_start.....	25
9.3.2 Comprobar sesion (Seguridad de pagina)	25
9.3.3 Destruir la sesión	25
9.4 \$_COOKIE.....	25
9.5 \$_SERVER.....	26
9.6 \$_FILES.....	26
9.7 \$_REQUEST	26
9.8 \$_ENV	26
9.9 \$GLOBALS	27
9.10 Operador null coalescing (??).....	27

APUNTES COMPLETOS DE PHP

10. IMPORTS (Inclusión de archivos)	28
10.1 Require	28
10.2 Include	28
10.3 Require_once	29
10.4 include_once	29
10.5 Buenas practicas	29
11. Formularios y su seguridad.....	30
11.1 Seguridad XSS (Cross-Site Scripting).....	30
11.2 Validar ≠ Sanitizar	30
11.2.1 Validar.....	30
11.2.1 Sanitizar	30
11.3 Flujo de como se comunica el formulario con el php	31
12. MySQL con PDO (conexión y CRUD).....	32
12.0 Conexión a la base de datos	32
12.1 Manejo de errores (IMPORTANTE).....	32
12.2 Consulta SELECT sin parámetros	33
12.3 Consulta con parámetros	33
12.4 INSERT	33
12.5 UPDATE.....	34
12.6 DELETE	34
13. Oracle con OCI8 (conexión y consultas)	35
13.0 Conexión Oracle	35
13.1 Consulta SELECT sin parámetros	35
13.2 Consulta SELECT con parámetros.....	36
13.3 INSERT	36
13.4 UPDATE.....	36
13.5 DELETE	36
13.7 Cerrar conexión	36
14. Errores y debugging.....	37
14.1 Mostrar errores en Desarrollo.....	37

APUNTES COMPLETOS DE PHP

14.2 Tipos de errores.....	37
14.3 Debugging básico.....	38
15. Programación Orientada a Objetos.....	39
15.1 Visibilidad	39
15.2 Crear clase	39
15.3 Atributos.....	39
15.4 Constructor.....	40
15.5 Getters y setters	40
15.5.1 Getter	40
15.1.2 Setter	40
15.6 Instanciar Objetos	40
15.7 Métodos de clase	40
15.8 Clases abstractas	41
15.9 Herencia.....	41
16. Estructura general de proyectos	42

APUNTES COMPLETOS DE PHP

0. Definición, instalación, configuración y ejecución de PHP

PHP es:

- **Lenguaje interpretado** (no se compila)
- **Orientado a servidor**
- **Request → Response**

Cada vez que el navegador llama a un .php, PHP:

1. Arranca
2. Ejecuta el archivo
3. Devuelve HTML / texto
4. Muere

No mantiene estado salvo que se use:

- sesiones
- cookies
- base de datos

PHP puede ejecutarse por línea de comandos o mediante un servidor web.

0.1 Windows (XAMPP)

Descargar XAMPP desde Apache Friends.

Instalar Apache y PHP.

Colocar archivos .php en htdocs.

Acceder mediante `http://localhost/archivo.php`.

0.2 Linux

```
sudo apt update
```

```
sudo apt install php apache2
```

```
php -v
```

`apt update` actualiza repositorios.

`apt install` instala PHP y Apache.

`php -v` muestra la versión.

APUNTES COMPLETOS DE PHP

0.3 Ejecución por terminal

`php` `archivo.php`

`php` invoca el intérprete.

`archivo.php` es el script a ejecutar.

La salida se muestra en consola.

1. Sintaxis PHP

```
<?php
echo "Hola mundo";
?>
```

Explicación línea por línea

`<?php` inicia el intérprete PHP.

`echo` envía salida al navegador.

`"Hola mundo"` es un string.

`;` finaliza la instrucción.

`?>` cierra el Bloque.

1.1 Cierre de PHP

```
<?php
echo "Hola";
```

Nunca se cierra `?>` si el archivo es solo PHP

Evita errores invisibles (espacios, BOM, headers)

1.2 Mezclar PHP y HTML

```
<h1>Hola</h1>
```

```
<?php
$nombre = "Egoitz";
?>
```

```
<p>Hola <?= $nombre ?></p>
```

```
<p>Bienvenido <?= $nombre ?></p>
```

`<?= ?>` es equivalente a `<?php echo ?>`

APUNTES COMPLETOS DE PHP

2. Comentarios

```
// comentario  
# comentario  
/* multilínea */
```

// comenta una línea.

alternativa de una línea.

/* */ comenta varias líneas.

3. Variables y tipos de datos

Las variables empiezan por \$.

PHP es de tipado dinámico.

El tipo se infiere automáticamente.

Guardan datos en memoria

3.1 Variables simples

```
$nombre = "Egoitz";  
$edad = 21;
```

3.2 Tipos de Datos

Tipo	Descripción	Ejemplo
string	Texto	"Hola"
int	Entero	10
float	Decimal	3.14
bool	Booleano	true
array	Colección	[1,2,3]
Null	Sin valor	null
object	Objeto	new Clase()

3.2.1 Ejemplo completo

Variable	Tipo
\$nombre="Egoitz";	string
\$edad=21;	int
\$nota=8.5;	float
\$activo=true;	bool
\$datos=[1,2,3];	array
\$nulo = null;	null
\$objeto = new Clase();	object

APUNTES COMPLETOS DE PHP

3.3.2 Strings

3.3.2.1 Comillas simples vs dobles

```
$nombre = "Egoitz";
```

```
echo "Hola $nombre";
```

Cuando PHP ve una cadena con **comillas dobles**:

1. Analiza el texto
2. Busca **variables** (\$algo)
3. Sustituye la variable por su **valor**
4. Construye el string final

```
echo 'Hola $nombre';
```

Cuando PHP ve **comillas simples**:

- NO busca variables
- NO sustituye nada
- Imprime el texto **tal cual está escrito**

3.3.2.2 Concatenación correcta

Esto **funciona siempre**, con cualquier comilla:

```
echo "Hola " . $nombre;
```

O sino también con llaves:

```
echo "Hola {$nombre}";
```

APUNTES COMPLETOS DE PHP

3.3.1.3 Funciones clave de los strings

Función	Descripción	Ejemplo	Resultado
strlen(\$texto);	Longitud (Cuenta todos los caracteres Incluye letras, espacios y símbolos)	\$texto = "Hola"; echo strlen(\$texto);	4
trim(\$texto);	Elimina espacios solo al principio y al final No toca los espacios del medio	\$texto = " Hola Mundo PHP "; echo trim(\$texto);	Hola Mundo PHP
strtolower(\$texto);	Pasa a minúsculas	\$texto = "Hola Mundo PHP"; echo strtolower(\$texto)	hola mundo php
strtoupper(\$texto);	Pasa a mayúsculas	\$texto = "Hola Mundo PHP"; echo strtoupper(\$texto);	HOLA MUNDO PHP
substr(\$texto, 0, 5);	Recortar un texto substr(texto, inicio, longitud); 0 → posición inicial 5 → número de caracteres	\$texto = "Hola Mundo PHP"; echo substr(\$texto, 0, 5);	Hola

APUNTES COMPLETOS DE PHP

3.4.2 Tipado Estricto

SIEMPRE en la primera línea del archivo

```
<?php
declare(strict_types=1);
```

Activa el modo estricto de tipos en ese archivo PHP.

- Obliga a que los tipos de datos que declares en funciones y retornos se respeten exactamente.
- Evita que PHP haga conversiones automáticas de tipos.

3.4.2.1 Ejemplo SIN strict_types

```
function sumar(int $a, int $b): int {
    return $a + $b;
}
echo sumar(5, "10");
```

RESULTADO -> 15

PHP convierte automáticamente "10" (string) a 10 (int).

Esto es **tipado débil**.

3.4.2.1 Ejemplo CON strict_types

```
<?php
declare(strict_types=1);
function sumar(int $a, int $b): int {
    return $a + $b;
}
echo sumar(5, "10");
```

RESULTADO -> FATAL ERROR

PHP **NO convierte** el string "10"

Lanza un error inmediatamente

APUNTES COMPLETOS DE PHP

5. Operadores

5.1 Operadores Aritméticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	echo 10 + 5;	15
-	Resta	echo 10 - 5;	5
*	Multiplica	echo 10 * 5;	50
/	Divide	echo 10 / 5	2
%	Resto de la división	echo 10 % 3	1

5.2 Operadores de Comparación

Operador	Descripción	Ejemplo	Resultado
==	Compara solo valor	10 == "10"	true
===	Compara valor y tipo	10 === "10"	false
!=	Distinto (valor)	10 != "10"	false
!==	Distinto (valor o tipo)	10 !== "10"	true

5.3 Operadores lógicos

Operador	Descripción	Ejemplo	Resultado
&&	AND (y)	true \$\$ false	false
`	OR	`	OR(o)
!	NOT (negación)	!true	false

5.4 Operadores relacionales

Operador	Descripción	Ejemplo	Resultado
<	Menor que	5 < 10	true
>	Mayor que	10 > 5	true
<=	Menor o igual que	5 <= 5	true
>=	Mayor o igual que	10 >= 5	true

5.5 Operador spaceship (PHP Moderno)

Operador	Descripción	Ejemplo	Resultado
<=>	Comparación triple	\$a <=> \$b;	-1 si \$a < \$b 0 si iguales 1 si \$a > \$b

APUNTES COMPLETOS DE PHP

5.6 Operadores de asignación combinada

Operador	Descripción	Ejemplo	Resultado
+=	Suma y asigna	<code>\$a = 10; \$a += 5;</code>	\$a vale 15
-=	Resta y asigna	<code>\$a = 10; \$a -= 3;</code>	\$a vale 7
*=	Multiplica y asigna	<code>\$a = 10; \$a *= 2;</code>	\$a vale 20
/=	Divide y asigna	<code>\$a = 10; \$a /= 2;</code>	\$a vale 5

5.7 Operador de concatenación

Operador	Descripción	Ejemplo	Resultado
.	Une strings	<code>"Hola " . "PHP"</code>	Hola PHP

5.8 Operador null coalescing

Operador	Descripción	Ejemplo	Resultado
??	Devuelve el primer valor que exista y no sea null	<code>\$_POST['nombre'] ?? "Invitado"</code>	"Invitado" si no existe

APUNTES COMPLETOS DE PHP

5. Estructuras de control

5.1 if / else

Es la **estructura de control más básica** para tomar decisiones.

- Ejecuta **un bloque u otro, nunca ambos**.

```
if ($edad >= 18) {  
    echo "Mayor";  
} else {  
    echo "Menor";  
}
```

1. PHP evalúa la condición:

```
$edad >= 18
```

2. Esa expresión devuelve true o false
3. Si es true → entra en el bloque if
4. Si es false → entra en el bloque else

Solo uno de los bloques se ejecuta.

La condición **siempre debe devolver un boolean**

Puedes usar:

- operadores de comparación (>=, ==, ===)
- operadores lógicos (&&, ||, !)

Se evalúa **de arriba abajo**

¿Cándo usar if / else?

Cuando tienes **condiciones complejas**

Cuando usas **rangos**, &&, ||

Cuando la lógica **no es solo igualdad**

APUNTES COMPLETOS DE PHP

5.2 switch

Es una alternativa a **muchos if / elseif** cuando:

- Comparas **la misma variable**
- Contra **muchos valores concretos**

```
switch ($rol) {  
    case 'admin':  
        echo "Panel admin";  
        break;  
    case 'user':  
        echo "Panel usuario";  
        break;  
    default:  
        echo "Sin permisos";  
}
```

1. PHP evalúa **una sola vez** la expresión:

`$rol`

2. Compara su valor con cada case
3. Cuando encuentra un case que coincide:
 - Ejecuta ese bloque
 - **Sigue ejecutando los siguientes** hasta encontrar un break

El break **es obligatorio** para evitar errores.

¿Cuándo usar switch?

- Cuando comparas **una variable contra muchos valores**
- Cuando todos los casos son **igualdad simple**
- Código legacy o PHP < 8

APUNTES COMPLETOS DE PHP

5.3 match (PHP 8+ -- alternativa a switch)

Es una **expresión**, no una estructura clásica.

- **Devuelve un valor**
- Se puede asignar directamente a una variable
- Mas seguro que switch
- No permite errores silenciosos

```
$resultado = match ($rol) {  
    'admin' => 'Panel admin',  
    'user' => 'Panel usuario',  
    default => 'Sin permisos',  
};
```

Diferencias frente a switch

1. Comparación estricta

Match usa ===, no ==

```
match (1) {  
    "1" => "texto", // NO coincide  
};
```

Evita errores silenciosos

2. No necesitas break

Cada rama ejecuta **solo una expresión**

Nunca hay *fall-through*

3. Debe ser exhaustivo

Si no hay coincidencia y no hay default -> ERROR

UnhandledMatchError

Detecta errores pronto

¿Cuándo usar match?

- En versiones de PHP 8 o superior
- Cuando se busca **código seguro**
- Cuando se **retornan valores**
- Cuando NO se quieran errores silenciosos

APUNTES COMPLETOS DE PHP

6. Bucles

6.1 for

Un bucle **controlado por contador**.

- Se usa cuando **sabes de antemano cuántas veces** quieres repetir algo.

```
for ($i = 0; $i < 5; $i++) {  
    echo $i;  
}
```

for (inicialización; condición; incremento)

1. Inicialización

```
$i = 0
```

Se ejecuta **una sola vez**

Crea el contador \$i

Marca el punto de inicio

2. Condición

```
$i < 5
```

Se evalúa **antes de cada vuelta**

Si es true → entra al bucle

Si es false → sale del bucle

3. Incremento

```
$i++
```

Se ejecuta **al final de cada iteración**

Evita bucles infinitos

¿Cuándo usar for?

- Contadores
- Repeticiones fijas
- Índices numéricos

APUNTES COMPLETOS DE PHP

6.2 While

Un bucle **condicional**.

Se repite **mientras la condición sea verdadera**.

- **NO sabes cuántas veces se repetirá**, solo sabes **cuándo debe parar**.

```
$i = 0;
while ($i < 5) {
    $i++;
}
```

1. PHP evalúa la condición:
 \$i < 5
2. Si es true → entra
3. Ejecuta el bloque
4. Vuelve a comprobar la condición
5. Repite hasta que sea false

Si la condición es false al inicio → **no entra ni una vez**.

¿Cuándo usar while?

- Lectura de datos
- Formularios
- Procesos que dependen de una condición
- Bases de datos (fetch)

Ejemplo:

```
while ($fila = $stmt->fetch()) {
    echo $fila['nombre'];
}
```

APUNTES COMPLETOS DE PHP

6.3 foreach

El bucle **especial para arrays y colecciones**.

- Recorre el array **de principio a fin automáticamente**
- **No necesitas contadores**

```
$colores = ["rojo", "verde"];  
foreach ($colores as $color) {  
    echo $color;  
}
```

Funcionamiento:

- PHP toma el array
- En cada vuelta:
 - Asigna el valor actual a \$color
- Continúa hasta el último elemento

No se usar foreach con variables que **no sean arrays u objetos**.

Foreach con clave y valor (lo que más se usa)

```
$usuario = [  
    "nombre" => "Egoitz",  
    "edad" => 21  
];  
  
foreach ($usuario as $clave => $valor) {  
    echo "$clave: $valor<br>";  
}
```

Salida:

```
nombre: Egoitz  
edad: 21
```

¿Cuándo usar foreach?

- Arrays
- Resultados de BD
- Listas
- JSON decodificado

APUNTES COMPLETOS DE PHP

7. Arrays

7.1 Sintaxis

Descripcion	Array	Ejemplo	Resultado
Array simple (Indexado)	<code>\$a = [dato, dato];</code>	<code>\$numeros = [1, 2, 3]; echo \$numeros[0];</code>	1
Array mezclado	<code>\$a = ["dato", dato];</code>	<code>\$arboles = ["roble", 2, true]; echo \$arboles[0];</code>	"roble"
Array Asociativo	<code>\$a = ["clave" => "valor"];</code>	<code>\$usuario = ["nombre"=>"Egoitz", " edad"=>21]; echo \$usuario["nombre"];</code>	Egoitz
Array de arrays	<code>\$a = [["clave" => "valor"], ["clave" => "valor"]];</code>	<code>\$usuarios = [['nombre'=>'Egoitz','edad'=>21], ['nombre'=>'Ana','edad'=>30]]; echo \$usuarios[0]['nombre'];</code>	Egoitz

APUNTES COMPLETOS DE PHP

7.2 Funciones del array

Función	Descripción	Ejemplo	Resultado
count(\$array)	Cuenta elementos	count([1,2,3])	3
in_array(\$valor, \$array)	Comprueba si existe un valor	in_array(2,[1,2,3])	true
array_keys(\$array)	Devuelve las claves	array_keys(['a'=>1,'b'=>2])	['a','b']
array_values(\$array)	Devuelve los valores	array_values(['a'=>1,'b'=>2])	[1,2]
array_merge(\$a,\$b)	Une arrays	array_merge([1,2],[3,4])	[1,2,3,4]
isset()	Comprueba si una clave existe y no es null	isset(\$user['email'])	false
array_key_exists()	Comprueba si una clave existe (aunque sea null)	array_key_exists('edad',\$user)	true
array_search()	Devuelve la clave de un valor	array_search(21,\$user)	'edad'
is_array()	Comprueba si es un array	is_array(\$user)	true
array_push()	Añade elementos al final	array_push(\$a,4)	[1,2,3,4]
array_pop()	Elimina el último elemento	array_pop(\$a)	elimina 3
array_shift()	Elimina el primer elemento	array_shift(\$a)	elimina 1
array_unshift()	Añade elementos al inicio	array_unshift(\$a,0)	[0,1,2,3]
unset()	Elimina una posición concreta	unset(\$a[1])	índice eliminado
array_map()	Aplica función a cada elemento	array_map('strtoupper',\$a)	valores en mayúsculas
array_filter()	Filtra según condición	array_filter(\$a)	elimina false/null
array_reduce()	Reduce a un solo valor	array_reduce(\$a,'sumar')	valor único
array_column()	Extrae una columna	array_column(\$users,'nombre')	['Egoitz','Ana']
sort()	Ordena valores (pierde claves)	sort(\$a)	valores ordenados
rsort()	Ordena valores inverso	rsort(\$a)	orden descendente

APUNTES COMPLETOS DE PHP

asort()	Ordena valores manteniendo claves	asort(\$a)	claves intactas
ksort()	Ordena por claves	ksort(\$a)	claves ordenadas
krsort()	Ordena claves inverso	krsort(\$a)	claves descendentes
array_slice()	Extrae parte del array	array_slice(\$a,1,2)	subarray
array_splice()	Elimina/reemplaza parte	array_splice(\$a,1,1)	elemento eliminado
array_merge_recursive()	Une arrays recursivamente	array_merge_recursive(\$a,\$b)	arrays combinados
array_combine()	Crea array clave→valor	array_combine(\$k,\$v)	array asociativo
array_unique()	Elimina duplicados	array_unique(\$a)	valores únicos
array_reverse()	Invierte el array	array_reverse(\$a)	orden inverso
array_diff()	Diferencia entre arrays	array_diff(\$a,\$b)	valores distintos
array_intersect()	Intersección	array_intersect(\$a,\$b)	valores comunes
array_rand()	Clave aleatoria	array_rand(\$a)	índice random

APUNTES COMPLETOS DE PHP

8. Funciones

Una **función** es un bloque de código reutilizable que:

- Se define **una vez**
- Se puede ejecutar **todas las veces que quieras**
- Sirve para **organizar, reutilizar y mantener** el código

8.1 Función básica

La función no se ejecuta sola, solo cuando se llama.

```
function saludar() {  
    echo "Hola";  
}  
Imprime: Hola
```

8.2 Parámetros y retorno sin tipado

Sin tipado, PHP **acepta cualquier tipo** (int, string, float...).

Esto es flexible, pero puede ocultar errores.

```
function sumar($a, $b) {  
    return $a + $b;  
}
```

\$a, \$b → parámetros de entrada

return → devuelve un resultado

8.3 Parámetros y retorno con tipado

```
function sumar(int $a, int $b): int {  
    return $a + $b;  
}
```

int \$a, int \$b → solo acepta enteros

: int → obliga a devolver un entero (también puede ser void, string...)

APUNTES COMPLETOS DE PHP

8.4 Funciones anónimas(closures)

```
$saludo = function($nombre) {  
    return "Hola $nombre";  
};
```

- **No tienen nombre**
- Se asignan a una variable
- Se pueden pasar como argumentos
- Se usan mucho en callbacks y arrays

APUNTES COMPLETOS DE PHP

9. Superglobales

Las **superglobales** son **variables predefinidas de PHP** que:

- Están **siempre disponibles**, en cualquier archivo, función o clase
- **No necesitan global** para usarse
- Contienen datos del **usuario**, del **servidor** o del **entorno**
- **TODOS** son **arrays asociativos**

Variable	Uso	Ejemplo
<code>\$_GET</code>	Datos enviados por URL	<code>\$_GET['id']</code>
<code>\$_POST</code>	Datos de formularios (POST)	<code>\$_POST['nombre']</code>
<code>\$_SESSION</code>	Datos persistentes por sesión	<code>\$_SESSION['usuario']</code>
<code>\$_COOKIE</code>	Datos almacenados en cookies	<code>\$_COOKIE['tema']</code>
<code>\$_SERVER</code>	Información del servidor y petición	<code>\$_SERVER['REQUEST_METHOD']</code>
<code>\$_FILES</code>	Subida de archivos	<code>\$_FILES['foto']['name']</code>
<code>\$_REQUEST</code>	Mezcla GET + POST + COOKIE	<code>\$_REQUEST['nombre']</code>
<code>\$_ENV</code>	Variables de entorno	<code>\$_ENV['PATH']</code>
<code>\$GLOBALS</code>	Acceso a variables globales	<code>\$GLOBALS['total']</code>

9.1 `$_GET`

- Datos visibles en la URL
- Ideal para filtros, IDs, paginación
- No usar para datos sensibles

Ejemplo:

```
// URL: pagina.php?id=5
$id = $_GET['id'] ?? null;
```

9.2 `$_POST`

- Datos enviados en el cuerpo de la petición
- Usado en formularios
- Más seguro que GET

Ejemplo:

```
$nombre = $_POST['nombre'] ?? '';
```

APUNTES COMPLETOS DE PHP

9.3 \$_SESSION

- Almacena datos mientras dura la sesión del usuario
- Requiere session_start()
- Vive en el servidor, no en el navegador

Ejemplo:

```
session_start();
$_SESSION['usuario'] = 'Egoitz';
```

9.3.1 Posicionar session_start

```
<php
    session_start
...

```

SIEMPRE tiene que ir en la primera línea del archivo .php

9.3.2 Comprobar sesión (Seguridad de página)

```
if (!isset($_SESSION['usuario'])) {
    header("Location: login.php");
    exit;
}
```

9.3.3 Destruir la sesión

```
session_destroy();
```

Hace que se cierre la sesión.

9.4 \$_COOKIE

- Datos guardados en el navegador
- Persisten incluso cerrando el navegador

Ejemplo:

```
setcookie('idioma', 'es', time()+3600);
echo $_COOKIE['idioma'];
```

APUNTES COMPLETOS DE PHP

9.5 \$_SERVER

- Información técnica de la petición

Claves comunes:

- \$_SERVER['REQUEST_METHOD']
- \$_SERVER['HTTP_USER_AGENT']
- \$_SERVER['SCRIPT_NAME']

Ejemplo:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    // procesar formulario  
}
```

9.6 \$_FILES

- Gestiona la subida de archivos
- Contiene información como: name, type, tmp_name, size, error
- Requiere formulario con enctype="multipart/form-data"
- Siempre validar tipo y tamaño

Ejemplo:

```
// <form method="post" enctype="multipart/form-data">  
$archivo = $_FILES['foto'];  
$nombre = $archivo['name'];  
$tmp     = $archivo['tmp_name'];
```

9.7 \$_REQUEST

- Mezcla datos de \$_GET, \$_POST y \$_COOKIE
- No recomendada por seguridad
- Puede provocar ambigüedades

Ejemplo:

```
$nombre = $_REQUEST['nombre'] ?? 'Invitado';
```

9.8 \$_ENV

- Contiene variables del sistema y del entorno
- Útil para configuraciones (entorno, claves, flags)
- Su disponibilidad depende del servidor

Ejemplo:

```
$entorno = $_ENV['APP_ENV'] ?? 'production';
```

APUNTES COMPLETOS DE PHP

9.9 \$GLOBALS

- Array que contiene todas las variables globales
- Permite acceder a variables globales dentro de funciones
- Poco usada en código moderno
- Mejor pasar parámetros siempre que sea posible

Ejemplo:

```
$total = 10;
function sumar() {
    $GLOBALS['total'] += 5;
}
sumar();
// $total vale 15
```

9.10 Operador null coalescing (??)

- Devuelve el **primer valor que exista y no sea null**
- Si no existe → usa el valor por defecto
- No lanza *warnings* por índices inexistentes

Internamente funciona como un `isset()` seguro.

```
$nombre = $_POST['nombre'] ?? 'Invitado';
```

Equivalencia:

```
if (isset($_POST['nombre'])) {
    $nombre = $_POST['nombre'];
} else {
    $nombre = 'Invitado';
}
```

El operador `??` simplifica este patón

APUNTES COMPLETOS DE PHP

10. IMPORTS (Inclusión de archivos)

Los *imports* permiten **dividir el código en archivos**, reutilizarlo y **organizar mejor los proyectos**.

Se usan para incluir:

- configuraciones
- cabeceras y pies HTML
- funciones comunes
- conexión a base de datos

10.1 Require

- Incluye y ejecuta el archivo indicado
- Si el archivo **NO existe** o hay un error: **PHP detiene la ejecución del script** (*error fatal*)

```
require 'config.php';
```

¿Cuándo usar **require**?

- Archivos **imprescindibles**
- Configuración (config.php)
- Conexión a base de datos
- Funciones críticas

10.2 Include

- Incluye y ejecuta el archivo indicado
- Si el archivo **NO existe**: PHP muestra un *warning*, **pero el script continúa**

```
include 'header.php';
```

¿Cuándo usar **include**?

- Archivos **no críticos**
- Cabeceras HTML
- Menús
- Footer

APUNTES COMPLETOS DE PHP

10.3 Require_once

- Igual que require
- **Evita incluir el mismo archivo más de una vez**

```
require_once 'funciones.php';
```

¿Cuándo usar require_once?

- Archivos con funciones
- Clases
- Configuración global

10.4 include_once

- Igual que include
- Solo incluye el archivo una vez

```
include_once 'menu.php';
```

¿Cuándo usar include_once?

- Componentes HTML reutilizables
- Plantillas

10.5 Buenas practicas

Usar rutas claras

```
Require _DIR_ . '/config/config.php';
```

Evita problemas de rutas relativas.

APUNTES COMPLETOS DE PHP

11. Formularios y su seguridad

11.1 Seguridad XSS (Cross-Site Scripting)

Un ataque XSS consiste en **inyectar código HTML o JavaScript** en una página para que se ejecute en el navegador de otros usuarios.

```
htmlspecialchars($_POST["nombre"]);
```

- Convierte caracteres peligrosos en texto seguro
- Evita que el navegador interprete HTML o JavaScript
- Protege contra ataques XSS

11.2 Validar ≠ Sanitizar

11.2.1 Validar

Comprueba si el dato es correcto

```
filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
```

- Comprueba si el email es valido
- Devuelve false si no cumple el formato

Se usa para decidir si el dato es aceptable

11.2.1 Sanitizar

Limpia el dato

```
$email = filter_input(INPUT_POST, 'email', FILTER_SANITIZE_EMAIL);
```

- Elimina caracteres no permitidos
- No garantiza que el dato sea válido

Se usa para limpiar, no para validar

APUNTES COMPLETOS DE PHP

11.3 Flujo de como se comunica el formulario con el php

HTML (Cliente):

```
<form method="post">
    <input name="nombre">
    <button type="submit">Enviar</button>
</form>
```

- El usuario introduce datos
- El navegador envía los datos al servidor
- El nombre del campo (name) será la clave en \$_POST

PHP (Servidor):

```
$nombre = $_POST["nombre"] ?? '';
```

- PHP recibe los datos en \$_POST
- ?? evita errores si el campo no existe

11.4 Comprobación del método

Se comprueba como llegó al formulario

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre = htmlspecialchars($_POST['nombre'] ?? '');
}
```

Evita procesar datos cuando alguien entra directamente a la URL

APUNTES COMPLETOS DE PHP

12. MySQL con PDO (conexión y CRUD)

PDO (PHP Data Objects) es la forma **recomendada y moderna** de trabajar con bases de datos en PHP:

- Soporta múltiples motores (MySQL, PostgreSQL, SQLite...)
- Usa **consultas preparadas**
- Protege contra **SQL Injection**

12.0 Conexión a la base de datos

```
$pdo = new PDO(
    "mysql:host=localhost;dbname=bd;charset=utf8",
    "user",
    "pass"
);
```

- Crea la conexión con MySQL
- charset=utf8 evita problemas con acentos y caracteres especiales

Práctica moderna: usar utf8mb4 si es posible (soporta emojis y más idiomas).

12.1 Manejo de errores (IMPORTANTE)

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

- Convierte errores SQL en **excepciones**
- Permite capturarlos con try / catch
- Evita errores silenciosos

Forma correcta de conexión

```
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=bd;charset=utf8",
        "user",
        "pass"
    );
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    echo $e->getMessage();
}
```

APUNTES COMPLETOS DE PHP

12.2 Consulta SELECT sin parámetros

```
$sql = "SELECT * FROM usuarios";  
$stmt = $pdo->prepare($sql);  
$stmt->execute();  
$stmt->fetchAll();  
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

- prepare() prepara la consulta
- execute() la ejecuta
- fetchAll() devuelve todos los registros

PDO::FETCH_ASSOC devuelve arrays asociativos (más limpio).

12.3 Consulta con parámetros

```
$sql = "SELECT * FROM usuarios WHERE email = :email";  
$stmt = $pdo->prepare($sql);  
$stmt->bindParam(':email', $email);  
$stmt->execute();  
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

- Protege contra **SQL Injection**
- Separa SQL de los datos
- Más legible y seguro

Alternativa mas corta:

```
$stmt->execute(['email' => $email]);
```

12.4 INSERT

```
$sql = "INSERT INTO usuarios (nombre, email) VALUES (:nombre, :email)";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([  
    'nombre' => $nombre,  
    'email'   => $email  
]);
```

El ultimo id generado se obtiene con:

```
$ultimoId = $pdo->lastInsertId();
```

APUNTES COMPLETOS DE PHP

12.5 UPDATE

```
$sql = "UPDATE usuarios SET nombre = :nombre WHERE id = :id";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([  
    'nombre' => $nombre,  
    'id'      => $id  
]);
```

Para saber cuantas filas se modificaron:

```
$filas = $stmt->rowCount();
```

12.6 DELETE

```
$sql = "DELETE FROM usuarios WHERE id = :id";  
$stmt = $pdo->prepare($sql);  
$stmt->execute(['id' => $id])
```

APUNTES COMPLETOS DE PHP

13. Oracle con OCI8 (conexión y consultas)

OCI8 es la extensión oficial de PHP para trabajar con **bases de datos Oracle**.

Permite ejecutar consultas SQL, usar parámetros enlazados y gestionar transacciones de forma segura.

Requiere que:

- Oracle Client esté instalado
- La extensión oci8 esté habilitada en PHP

13.0 Conexión Oracle

```
$conn = oci_connect(  
    "usuario",  
    "password",  
    "localhost:1521/XEPDB1"  
);
```

- oci_connect() abre una conexión con Oracle
- 1521 → puerto por defecto de Oracle
- XEPDB1 → servicio / base de datos

Si la conexión falla, devuelve false.

Comprobar la conexión:

```
if (!$conn) {  
    $error = oci_error();  
    die($error['message']);  
}
```

13.1 Consulta SELECT sin parámetros

```
$sql = "SELECT * FROM usuarios";  
$stmt = oci_parse($conn, $sql);  
oci_execute($stmt);  
  
while ($fila = oci_fetch_assoc($stmt)) {  
    echo $fila['NOMBRE'];  
}
```

- oci_parse() prepara la consulta
- oci_execute() la ejecuta
- oci_fetch_assoc() obtiene filas como array asociativo

Oracle devuelve los nombres de columnas en **MAYÚSCULAS**.

APUNTES COMPLETOS DE PHP

13.2 Consulta SELECT con parámetros

```
$sql = "SELECT * FROM usuarios WHERE email = :email";  
$stmt = oci_parse($conn, $sql);  
oci_bind_by_name($stmt, ":email", $email);  
oci_execute($stmt);
```

- Evita **SQL Injection**
- Oracle separa SQL de datos
- Mejora seguridad y legibilidad

oci_bind_by_name() enlaza la variable **por referencia**.

13.3 INSERT

```
$sql = "INSERT INTO usuarios (nombre) VALUES (:nombre)";  
$stmt = oci_parse($conn, $sql);  
oci_bind_by_name($stmt, ":nombre", $nombre);  
oci_execute($stmt, OCI_COMMIT_ON_SUCCESS);
```

13.4 UPDATE

```
$sql = "UPDATE usuarios SET nombre = :nombre WHERE id = :id";  
$stmt = oci_parse($conn, $sql);  
oci_bind_by_name($stmt, ":nombre", $nombre);  
oci_bind_by_name($stmt, ":id", $id);  
oci_execute($stmt, OCI_COMMIT_ON_SUCCESS);
```

13.5 DELETE

```
$sql = "DELETE FROM usuarios WHERE id = :id";  
$stmt = oci_parse($conn, $sql);  
oci_bind_by_name($stmt, ":id", $id);  
oci_execute($stmt, OCI_COMMIT_ON_SUCCESS);
```

- OCI_COMMIT_ON_SUCCESS hace **commit automático**
- OCI_NO_AUTO_COMMIT **impide que haga el commit automático**.
- Sin OCI_COMMIT_ON_SUCCESS, los cambios **no se guardan** hasta llamar a oci_commit()

13.7 Cerrar conexión

```
oci_free_statement($stmt);  
oci_close($conn);
```

- Libera memoria
- Evita fugas de recursos

APUNTES COMPLETOS DE PHP

14. Errores y debugging

PHP dispone de un sistema de **gestión de errores** que permite:

- Detectar fallos de sintaxis
- Ver errores en tiempo de ejecución
- Depurar código de forma controlada
- Evitar mostrar información sensible al usuario final

14.1 Mostrar errores en Desarrollo

```
ini_set('display_errors', 1);  
ini_set('display_startup_errors', 1);  
error_reporting(E_ALL);
```

¿Qué hace cada línea?

```
ini_set('display_errors', 1);
```

- Muestra los errores en pantalla
- Ideal para **desarrollo y aprendizaje**

```
ini_set('display_startup_errors', 1);
```

- Muestra errores que ocurren **antes** de que el script empiece
- Muy útil para errores de configuración

```
error_reporting(E_ALL);
```

- Muestra **todos los errores, warnings y notices**
- Incluye errores deprecados (PHP moderno)

14.2 Tipos de errores

Tipo	Descripción
E_PARSE	Error de sintaxis (falta ;, {})
E_ERROR	Error fatal (detiene el script)
E_WARNING	Error no fatal (el script continúa)
E_NOTICE	Avisos menores (variables no definidas)
E_DEPRECATED	Funciones obsoletas
E_ALL	Todos los errores
Tipo	Descripción

APUNTES COMPLETOS DE PHP

14.3 Debugging básico

```
var_dump($variable);
```

- Muestra tipo y valor
- Ideal para depurar arrays y objetos

```
print_r($array);
```

- Más legible para arrays
- Menos detalle que var_dump

```
die() / exit()
```

```
die('Aquí llegó');
```

- Detiene la ejecución
- Útil para saber hasta dónde llega el código

APUNTES COMPLETOS DE PHP

15. Programación Orientada a Objetos

La **Programación Orientada a Objetos** organiza el código en **clases** que representan **entidades del mundo real** y permiten:

- Reutilizar código
- Encapsular datos
- Mantener proyectos grandes
- Trabajar en equipo

15.1 Visibilidad

Visibilidad	Uso
public	Accesible desde cualquier parte
protected	Accesible desde la clase y sus hijas
private	Solo accesible dentro de la propia clase
abstract	Clases o métodos que no se pueden instanciar

15.2 Crear clase

- Es un **molde**
- Define cómo serán los objetos
- No ejecuta nada por sí sola

```
class Usuario {  
    // contenido de la clase  
}
```

15.3 Atributos

- Se declaran dentro de la clase
- Se recomienda **tipar siempre**
- Los atributos **no deberían ser públicos**.

```
class Usuario {  
    private string $nombre;  
    private int $edad;  
    private bool $mayor;  
}
```

APUNTES COMPLETOS DE PHP

15.4 Constructor

```
class Usuario {  
  
    private string $nombre;  
  
    public function __construct(string $nombre) {  
        $this->nombre = $nombre;  
    }  
}
```

15.5 Getters y setters

15.5.1 Getter

```
public function getNombre(): string {  
    return $this->nombre;  
}
```

15.1.2 Setter

```
public function setNombre(string $nombre): void {  
    $this->nombre = $nombre;  
}
```

15.6 Instanciar Objetos

```
$u = new Usuario("Egoitz");  
echo $u->getNombre();
```

- Se crea un objeto Usuario
- Se ejecuta el constructor
- Se accede al método público

15.7 Métodos de clase

Los métodos representan **comportamientos** del objeto.

```
public function esMayorDeEdad(): bool {  
    return $this->edad >= 18;  
}
```

APUNTES COMPLETOS DE PHP

15.8 Clases abstractas

Una clase abstracta:

- No se puede instanciar
- Sirve como base para otras clases

```
abstract class Persona {  
    abstract public function saludar(): string;  
}
```

15.9 Herencia

Permite reutilizar código y especializar clases.

```
class Admin extends Usuario {  
    public function esAdmin(): bool {  
        return true;  
    }  
}
```

APUNTES COMPLETOS DE PHP

16. Estructura general de proyectos

Carpetas	Uso
/config	Contiene archivos de configuración del proyecto: conexión a base de datos, constantes globales, configuración de entorno. No debe ser accesible desde el navegador.
/controllers	Gestiona la lógica de control: recibe las peticiones del usuario, valida datos y decide qué modelo usar y qué vista mostrar. Es el intermediario entre modelos y vistas .
/models	Contiene la lógica de negocio y acceso a datos. Se encarga de consultar la base de datos, validar reglas de negocio y devolver datos estructurados.
/views	Contiene la capa de presentación (HTML + PHP). Su función es mostrar datos , no procesarlos. No debe contener lógica compleja.
/public	Carpeta pública del proyecto. Contiene el punto de entrada (index.php) y archivos estáticos (CSS, JS, imágenes). Es la única carpeta accesible desde el navegador .